MACHINE LEARNING PROJECT

Fashion-MNIST Image Classification Using Simple RNN

A deep learning project that classifies grayscale clothing images into ten categories using a Simple Recurrent Neural Network.

Project Detail	Information
Project Title	Fashion-MNIST Image Classification Using Simple RNN
Domain	Artificial Intelligence / Deep Learning / Computer Vision
Framework	TensorFlow and Keras
Platform	Google Colab / Python
Dataset	Fashion-MNIST (70,000 grayscale images)
Model Type	Simple Recurrent Neural Network (SimpleRNN)
Submitted By	Shafaat Ullah

Note: This document describes the implemented workflow. Final accuracy and other evaluation values must be filled in from the actual Google Colab run; no performance result is assumed.

1. Abstract

This project applies a Simple Recurrent Neural Network (SimpleRNN) to the Fashion-MNIST dataset for multi-class image classification. Fashion-MNIST contains 70,000 grayscale images of clothing items, each with a resolution of 28 × 28 pixels. The images are normalized and treated as sequences of 28 time steps, where each time step contains 28 pixel values. The model learns patterns from these sequences and predicts one of ten clothing categories. Performance is evaluated using test accuracy, loss, a confusion matrix, and a classification report.

2. Introduction

Image classification is a computer vision task in which a model assigns an image to one of several predefined categories. Fashion-MNIST is a commonly used benchmark dataset that is more challenging than handwritten digit recognition because several clothing categories have similar visual appearances. Although convolutional neural networks are generally a natural choice for image classification, this project uses a SimpleRNN to explore how recurrent models can process image rows as sequential input.

3. Project Objectives

- Load Fashion-MNIST directly through TensorFlow/Keras without manually downloading a ZIP file.
- Normalize image pixel values to improve numerical stability during training.
- Train a SimpleRNN-based neural network to classify images into ten categories.
- Track training and validation accuracy and loss.
- Evaluate generalization on the held-out test dataset.
- Visualize classification errors and compare actual labels with predictions.
- Export prediction results to CSV and save the trained model.

4. Dataset Description

Fashion-MNIST is distributed with TensorFlow/Keras and is also available through public dataset repositories. It includes 60,000 training images and 10,000 test images. Each image is 28 × 28 pixels with grayscale intensity values from 0 to 255. Each image has a label from 0 to 9.

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

The project splits the original 60,000-image training portion into training and validation subsets using a stratified 90/10 split. The official 10,000-image test set is kept separate for final evaluation.

5. Tools and Technologies

- Python: programming language.
- TensorFlow/Keras: model construction, training, and dataset loading.
- NumPy and Pandas: numerical operations and tabular prediction output.
- Matplotlib: sample image display and learning curves.
- Scikit-learn: stratified splitting, confusion matrix, accuracy, and classification report.
- Google Colab: cloud notebook environment for running the project.

6. Data Preprocessing

6.1 Pixel normalization

Each pixel intensity is converted from an integer in the range 0–255 to a floating-point value in the range 0–1 by dividing by 255.0. Normalization helps keep input values on a consistent scale.

```
normalized_pixel = pixel_value / 255.0
```

6.2 Sequence representation

Every 28 × 28 image is provided to the recurrent model as a sequence with 28 time steps and 28 features per time step. The RNN therefore processes each row as one step in the sequence. This representation is used for experimentation; it does not preserve image structure as effectively as a purpose-built CNN.

6.3 Train-validation-test split

The 60,000 original training examples are split into 54,000 training images and 6,000 validation images. The 10,000 official test images are reserved for final evaluation.

7. Model Architecture

The network is built with Keras Sequential layers. Its input shape is (28, 28), representing 28 time steps with 28 pixel values per step.

Layer	Configuration	Purpose
Input	28 time steps × 28 features	Accepts each image as a sequence
SimpleRNN 1	128 units, return_sequences=True	Learns sequential patterns across image rows
Dropout	0.2	Regularization to reduce overfitting
SimpleRNN 2	64 units	Produces a compact sequence representation
Dropout	0.3	Additional regularization
Dense	64 units, ReLU	Learns non-linear combinations of features
Dropout	0.2	Additional regularization
Output	10 units, Softmax	Outputs probabilities for the ten classes

Compilation settings: Adam optimizer, learning rate 0.001, sparse categorical cross-entropy loss, and accuracy as the training metric.

8. Model Training

The model is trained for up to 20 epochs with a batch size of 128. Validation data is monitored after each epoch. EarlyStopping monitors validation loss and restores the best observed weights if progress stalls. ReduceLROnPlateau lowers the learning rate when validation loss stops improving, which may help the model converge more smoothly.

- Maximum epochs: 20.
- Batch size: 128.
- Optimizer: Adam.
- Loss function: sparse categorical cross-entropy.
- Callbacks: EarlyStopping and ReduceLROnPlateau.
- Random seed: 42 for repeatable initialization and data splitting.

9. Evaluation Metrics

Accuracy

Accuracy is the proportion of test images whose predicted class matches the true label.

```
Accuracy = Correct Predictions / Total Predictions
```

Precision, Recall, and F1-score

Precision measures how many predictions for a class are correct. Recall measures how many true examples of a class are found. The F1-score is the harmonic mean of precision and recall. The classification report provides these metrics for each clothing class.

Confusion matrix

A confusion matrix shows true classes against predicted classes. Diagonal cells represent correct classifications, while off-diagonal cells show which classes the model confuses.

Loss

Training and validation loss curves help identify convergence and possible overfitting. If training loss continues to decrease while validation loss rises, the model may be overfitting.

10. Results

The exact test accuracy, test loss, and class-wise metrics depend on running the notebook. These values have intentionally not been fabricated. After executing the notebook, enter the actual values in the table below.

Metric	Actual result from Colab
Test accuracy	_____
Test loss	_____
Weighted precision	_____
Weighted recall	_____
Weighted F1-score	_____

Expected notebook outputs include accuracy/loss graphs, a confusion matrix, a classification report, example predictions, a CSV file, and a saved Keras model.

11. Output Files

- **fashion_mnist_rnn_predictions.csv** — actual labels, predicted labels, correctness, and confidence percentages for test images.
- **fashion_mnist_simple_rnn.keras** — saved trained model that can be loaded later for inference.

12. Limitations

- A SimpleRNN is not designed specifically for two-dimensional image structure, so it may perform worse than a CNN on Fashion-MNIST.
- Treating image rows as a sequence makes row order important and does not directly model both image dimensions in the same way a CNN does.
- Accuracy may vary with initialization, training time, library versions, and hardware.
- High confidence does not guarantee a correct prediction.
- This is an educational image-classification project, not a production-grade visual recognition system.

13. Future Improvements

- Compare SimpleRNN against a CNN baseline on the same train and test split.
- Tune the number of recurrent units, dropout rates, learning rate, and batch size.
- Use per-class metrics to identify difficult categories such as Shirt, T-shirt/top, Coat, and Pullover.
- Try a CNN-RNN hybrid or a more suitable image model and compare results fairly.
- Save confusion matrix and training curves as images for the project report.

14. Conclusion

This project demonstrates an end-to-end deep learning workflow for multi-class classification with Fashion-MNIST and a SimpleRNN model. It covers dataset loading, normalization, splitting, model construction, training, evaluation, visualization, prediction export, and model saving. The final performance should be reported using the actual results produced by the Colab notebook. The project also provides a useful baseline for comparing recurrent networks with CNNs on image data.

15. References

- Fashion-MNIST dataset: https://github.com/zalandoresearch/fashion-mnist
- TensorFlow Keras Fashion-MNIST API: TensorFlow Documentation
- Keras SimpleRNN layer: TensorFlow Documentation
- Scikit-learn metrics: Model Evaluation Guide